

▶ 3D 섯다 코드 해설서

(5살도 이해하는 눈높이 버전)

대상 파일: 3D_섯다.html (총 1,197줄) · 코드를 통째로 다 옮기지는 않고, 의미 있는 덩어리(블록)마다 줄 번호를 표시하고 그 블록이 "무엇을 하는지"를 아주 쉬운 말로 설명합니다.

▶ 이 파일 전체는 무엇인가요?

이 파일 하나가 **게임 전체**예요! 마치 장난감 상자 하나에 "레고 블록(그림 그리는 방법)", "설명서(규칙)", "로봇 친구(컴퓨터가 카드 내는 방법)"가 다 같이 들어있는 것과 같아요. 파일은 크게 두 부분으로 나뉘어요:

- **<style> 부분** (CSS): 화면을 예쁘게 꾸미는 "색칠 규칙"이에요. 버튼을 무슨 색으로 할지, 글씨를 어디에 놓을지 정해요.
- **<script> 부분** (JavaScript): 진짜 게임이 "어떻게 움직이는지" 정하는 두뇌예요. 카드를 섞고, 누가 이겼는지 정하고, 컴퓨터 친구가 무슨 생각을 하는지 다 여기서 정해요.

목차

1. 화면 꺾데기 - <head>와 기본 설정 (1~10줄)
2. 화면을 꾸미는 색칠 규칙 - <style> (7~206줄)
3. 눈에 보이는 진짜 화면 - <body> (211~286줄)
4. 카드 그림을 준비하는 곳 - RealisticHwatuRenderer (290~391줄)
5. 왼쪽 족보표를 그리는 곳 - JokboBoard (317~366줄)
6. 섯다 규칙을 아는 두뇌 - SeotdaEngine (369~417줄)
7. 입체 카드 마법사 - Renderer3D (419~555줄)
8. 소리를 만드는 마법사 - SoundFX (558~625줄)
9. 게임의 진짜 두뇌 - Game (627~1051줄)
10. 마지막 마무리와 보너스 코드 (1053~1197줄)

1. 화면 껍데기 - <head>와 기본 설정 1~10줄

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>3D 섯다</title>
```

- ▶ **<!DOCTYPE html>**: 컴퓨터한테 "이건 웹사이트 종이야!" 라고 맨 처음에 말해주는 거예요.
- <html lang="ko">**: "이 웹사이트는 통째로 한국어로 되어있어요" 라고 알려줘요.
- <head>**: 눈에는 안 보이지만 컴퓨터가 알아야 하는 "준비물 목록"이에요. 마치 요리하기 전에 재료를 미리 꺼내놓는 것과 같아요.
- <meta charset="UTF-8">**: "한글이나 이모지가 깨지지 않게 해줘!" 라는 부탁이에요.
- <meta name="viewport" ...>**: "핸드폰에서 봐도 화면 크기에 맞게 예쁘게 보여줘" 라는 부탁이에요.
- <title>3D 섯다</title>**: 브라우저 탭 위쪽에 적히는 이름표예요. "3D 섯다"라고 써져요.

2. 화면을 꾸미는 색칠 규칙 - <style> 7~206줄

이 부분은 전부 "이 이름표를 가진 물건은 이렇게 생겼어" 라고 정해주는 규칙이에요. 실제로 뭘 하지는 않고, 그냥 "생김새 설명서"예요. 장난감에 붙이는 스티커 설명서 같은 거예요.

2-1. 전체 배경과 기본 글씨체 8~14줄

```
* { box-sizing: border-box; user-select: none; }
body { margin: 0; padding: 0; overflow: hidden; background-color: #031208; ... color: #fff; }
#canvas-container { width: 100vw; height: 100vh; position: absolute; top: 0; left: 0; z-index: 1; }
```

- ▶ *** { ... }**: 별표(*)는 "이 페이지에 있는 모든 것"을 가리켜요. "모든 물건은 정해진 크기만 차지하고, 손가락으로 글씨를 드래그해서 선택 못하게 해줘"라고 정해요.
- body { ... }**: 이 웹페이지의 "몸통(전체 도화지)" 색깔을 아주 어두운 초록색(#031208, 카지노 테이블 같은 색)으로 칠하고, 글씨는 하얀색으로 정해요.
- #canvas-container**: 나중에 입체(3D) 카드 게임 그림이 그려질 "도화지 자리"를 화면 전체 크기로 잡아둬요.

2-2. 시작 화면 꾸미기 16~33줄

- ▶ 게임을 처음 켤 때 보이는 "게임 시작" 화면을 꾸미는 부분이에요. **#start-screen**은 화면 한가운데 글씨와 버튼을 세로로 가지런히 놓으라고 정해요. **h1**(제목 "3D 섯다")은 크고 노란색 글씨로, **.balance-info**(내 돈 표시)는 까만 알약 모양 배경에 놓이게 하고, **.bet-setting**(기본 판돈 고르는 곳)은 동그란 테두리 상자로 만드어요. **.btn-main**(게임 시작 버튼)은 빨간색-보라색이 섞인 예쁜 그라데이션 버튼으로, 마우스를 갖다 대면(:hover) 살짝 위로 뜨는 효과를 줘요.

2-3. 왼쪽 족보표, 상단바, 판돈 표시 36~68줄

- ▶ **.jokbo-panel**은 왼쪽 위에 떠 있는 "족보 설명서" 상자를 정해요(까만 반투명 배경, 노란 테두리). **.jokbo-item**은 그 안의 한 줄 한 줄(카드 그림 + 이름)이 어떻게 나열될지 정해요. **@media (max-width: 900px)**는 "화면이 좁아지면(핸드폰처럼) 족보표를 더 작게 만들어줘"라는 뜻이에요 — 마치 좁은 방에 들어가면 몸을 웅크리는 것과 같아요. **.top-bar**는 오른쪽 위 "내 잔고" 상자, **.pot-info**는 화면 맨 위 가운데 "지금 판돈이 얼마인지" 보여주는 노란 알약 상자예요.

2-4. "다음 판 기본 판돈" 설정 칸 70~205줄

- ▶ 이 부분은 꽤 길고 복잡한데(70~88줄, 151~205줄에 걸쳐 여러 번 다시 나와요), 전부 "다음 판 기본 판돈을 고르는 작은 상자"가 화면의 다른 상자들(족보표, 판돈 표시, AI 말풍선)과 겹치지 않게 자리를 잘 잡아주는 규칙이에요. **80~88줄**, **158~205줄**의 **@media**들은 "화면이 좁아지면 이 상자를 위로, 옆으로, 혹은 아예 다른 자리로 옮겨라"는 반복된 지시예요. 마치 방 크기가 바뀔 때마다 책상 위치를 조금씩 바꿔주는 것과 같아요. (실제 게임 규칙과는 상관없고, 오직 "안 겹치게" 하려는 화면 정리예요.)

2-5. AI 심리전 말풍선, 손패 이름표 90~113줄

- ▶ **#ai-psych-msg**는 상대(컴퓨터)가 "음... 콜 할게요" 같은 말을 하는 말풍선의 생김새예요. 평소엔 투명하다가(**opacity: 0**) 말을 할 때 스르륵 나타나도록(**.visible** 일 때 진해짐) 정해져 있어요. **.hand-label**은 내 패/상대 패의 이름(예: "알리", "장땡")을 보여주는 이름표 모양이에요.

▶ **.control-panel**은 화면 맨 아래 버튼들이 모인 검은 상자예요. **.bet-btn**은 버튼 기본 모양이고, 그 아래 **.call**(초록), **.half**(주황), **.quarter**(연주황), **.allin**(보라), **.die**(빨강)로 각 버튼마다 다른 색을 입혀서 "이건 콜 버튼, 이건 다이 버튼" 하고 한눈에 알아보게 해줘요. 신호등 색깔처럼요! **#custom-bet-modal**은 "직접 판돈 숫자를 입력하는 팝업창"의 생김새(화면 전체를 어둡게 가리고 가운데 하얀 상자를 띄움)를 정해요.

3. 눈에 보이는 진짜 화면 - <body> 211~286줄

여기부터는 앞에서 정한 "색칠 규칙"을 실제로 화면에 등장시키는 "물건들"이에요. 장난감 설명서 다음에 진짜 장난감이 나오는 것과 같아요.

3-1. 캔버스와 시작화면 213~232줄

```
<div id="canvas-container"></div>
<div id="start-screen" class="interactive">
  <h1>3D 섯다</h1>
  <div class="balance-info">보유 금액: <span id="start-balance">100,000</span> 원</div>
  ...
  <button class="btn-main" onclick="Game.startNewRound()">게임 시작</button>
</div>
```

▶ **#canvas-container**는 나중에 3D 카드와 테이블 그림이 그려질 빈 도화지예요. 지금은 텅 비어있어요.

#start-screen: 게임을 켜면 맨 처음 보이는 화면이에요. "3D 섯다"라는 제목, 내가 가진 돈(100,000원), 기본 판돈을 고르는 선택 상자, 그리고 **"게임 시작" 버튼**이 들어있어요. 이 버튼을 누르면 `Game.startNewRound()` 라는 "게임을 시작해줘!" 라는 주문이 실행돼요.

3-2. 게임 화면 안의 여러 상자들 235~267줄

▶ **#game-ui** 안에는 실제 게임을 하는 동안 보이는 모든 것이 들어있어요:

- **.jokbo-panel**: 왼쪽에 뜨는 족보 설명서 (JokboBoard가 나중에 내용을 채워요)
- **.pot-info**: 가운데 위 "판돈: 0 원"
- **#next-bet-setting**: "다음 판 기본 판돈"을 고르는 선택 상자
- **#ai-psych-msg**: 상대가 말하는 말풍선 (지금은 비어있음, "AI 심리전"이라는 제목만 있음)
- **.top-bar**: 오른쪽 위 "내 잔고: 100,000 원"
- **#ai-hand-label**, **#player-hand-label**: 내 패/상대 패 이름표 (지금은 `display:none` 으로 숨겨져 있음)
- **.control-panel**: 콜·하프·쿼터·올인·다이 다섯 개 버튼. 각 버튼을 누르면 `Game.handlePlayerBet('콜'/'하프'/'...')` 이라는 주문이 실행돼요.

3-3. 커스텀 판돈 팝업, 토스트 메시지 271~284줄

▶ **#custom-bet-modal**: "얼마로 할지 직접 써주세요"라는 팝업창이에요. 숫자 입력칸, "취소"/"적용" 버튼이 있어요.

#toast-msg: 화면 가운데 잠깐 나타났다 사라지는 알림 글씨예요 (예: "승리! +5,000원"). 평소엔 투명하다가 메시지가 있을 때만 반짝 보여요.

4. 카드 그림을 준비하는 곳 - RealisticHwatuRenderer 290~391줄

4-1. 어마어마하게 긴 암호문 (카드 그림 데이터) 293줄

▶ 이 파일에서 **딱 한 줄(293번째 줄)**이 **928,217글자**나 돼요! 책 한 권보다 길어요. 이걸 카드 20장의 진짜 사진을 "글자로 바꾼 암호문(base64 이미지)"이예요. 비유하자면, 그림을 아주아주 잘게 잘라서 "이 조각은 무슨 색, 저 조각은 무슨 색"을 글자로 하나하나 받아 적은 거예요. 컴퓨터는 이 긴 암호문을 다시 그림으로 썸! 하고 바꿔서 보여줄 수 있어요. 우리는 이 줄이 "카드 그림들이 숨어있는 보물상자"라고만 알면 돼요.

4-2. 카드 번호로 알맞은 그림 찾기 296~301줄

```
getCardImageIndex(cardData) {  
    const monthIndex = cardData.month - 1;  
    return (cardData.id % 2 === 1) ? monthIndex : monthIndex + 10;  
},  
createFrontTexture(cardData) {  
    return this.cardImages[this.getCardImageIndex(cardData)];  
},
```

▶ 카드마다 "몇 번째 달(月)인지"(1월~10월), 그리고 "짝수 번호 카드인지 홀수 번호 카드인지"를 보고, 보물상자 속 928,217글자 그림 중에서 **정확히 어느 조각을 꺼내와야 하는지**를 계산해요. 마치 사물함 번호를 보고 몇 번 칸을 열어야 하는지 아는 것과 같아요.

4-3. 카드 뒷면 그리기 303~314줄

▶ 카드 뒷면(글자가 안 보이는 쪽) 그림은 사진이 없어서, 컴퓨터가 **직접 빨간 네모를 그려서** 만들어요. 빨간 배경을 칠하고, 안쪽에 더 밝은 빨강 네모를 그리고, 줄무늬를 그어서 마치 진짜 화투 뒷면처럼 보이게 해요. 꼭 도화지에 자로 줄을 그어서 무늬를 만드는 것과 같아요.

5. 왼쪽 족보표를 그리는 곳 - JokboBoard 317~366줄

▶ 이 부분은 화면 왼쪽에 뜨는 "셋다 족보 설명서(어떤 카드 조합이 센지 보여주는 표)"를 자동으로 만들어줘요.

- **cardImage()**: 몇 월 카드 그림인지 찾아와요.
- **pair()**: 카드 두 장 그림을 나란히 놓는 작은 상자 하나를 만들어요.
- **row()**: 순위 숫자 + 카드 두 장 그림 + 이름(예: "장땡")을 한 줄로 만들어요.
- **section()**: "◆ 최상위 (광땡)" 같은 제목 줄을 만들어요.
- **render()**: 광땡 3개, 땡 10개, 중간족보 6개, 기본족보(끗) 10개를 **목록으로 미리 다 적어두고**, 하나씩 순서대로 화면에 그려 넣어요. 마치 선생님이 칠판에 "1등, 2등, 3등..." 순서대로 표를 그려나가는 것과 같아요.

6. 섯다 규칙을 아는 두뇌 - SeotdaEngine 369~417줄

이 부분이 게임에서 **가장 중요한 규칙** 담당이에요. "이 카드 두 장이 뭐가 되는지"를 정확히 판단해요.

6-1. 카드 20장 목록 376~391줄

```
cardsData: [
  { id: 1, month: 1, kwang: true }, { id: 2, month: 1, kwang: false },
  { id: 3, month: 2, kwang: false }, ...
  { id: 19, month: 10, kwang: false }, { id: 20, month: 10, kwang: false }
],
```

★ 화투 카드 20장을 하나하나 적어놓은 명단이에요. 각 카드는 세 가지 정보를 가져요: **id**(1번부터 20번까지 이름표), **month**(몇 월 카드인지, 1~10), **kwang**(반짝이는 "광" 카드인지 아닌지, 참/거짓). 1월, 3월, 8월에만 광 카드가 하나씩 숨어있어요(`kwang: true`).

6-2. evalHand: 카드 두 장을 보고 "이게 뭔지" 알아내기 393~416줄

```
evalHand(c1, c2) {
  const m1 = c1.month, m2 = c2.month;
  const k1 = c1.kwang, k2 = c2.kwang;

  if (k1 && k2) {
    if ((m1===3 && m2===8) || (m1===8 && m2===3)) return { score: 1000, name: "3·8 광땡" };
    if ((m1===1 && m2===3) || (m1===3 && m2===1)) return { score: 990, name: "1·3 광땡" };
    if ((m1===1 && m2===8) || (m1===8 && m2===1)) return { score: 990, name: "1·8 광땡" };
  }
  if (m1 === m2) return { score: 800 + m1, name: `${m1===10 ? '장' : m1}땡` };

  const pairStr = [m1, m2].sort((a,b)=>a-b).join(',');
  if (pairStr === "1,2") return { score: 750, name: "알리" };
  if (pairStr === "1,4") return { score: 740, name: "독사" };
  if (pairStr === "1,9") return { score: 730, name: "구빙" };
  if (pairStr === "1,10") return { score: 720, name: "장빙" };
  if (pairStr === "4,10") return { score: 710, name: "장사" };
  if (pairStr === "4,6") return { score: 700, name: "세륙" };

  const kkut = (m1 + m2) % 10;
  if (kkut === 0) return { score: 100, name: "망통" };
  return { score: 100 + kkut, name: `${kkut}끗` };
}
```

▶ 이 함수는 카드 두 장을 받아서 "너희 둘이 합쳐지면 뭐가 되는지"를 **순서대로 하나씩 검사**해요. 꼭 스무고개 놀이 같아요!

1. **1번째 질문** (397~401줄): "둘 다 반짝이는 광 카드야?" 그렇다면 어느 달인지 보고 "1000점짜리 3·8광땡"처럼 **제일 센 패**를 줘요. 이게 가장 먼저 물어보는 질문이라, 광땡이 제일 힘이 세요.
2. **2번째 질문** (403줄): "두 카드가 같은 달이야?" 맞으면 "땡"이에요. 숫자가 클수록(예: 10월=장땡) 더 세요.
3. **3번째 질문** (405~411줄): "혹시 정해진 특별 조합(1,2월 = 알리, 1,4월 = 독사 등)이야?" 여섯 가지 특별한 조합을 하나씩 확인해요.
4. **마지막** (413~415줄): 그것도 아니면, 두 달을 더해서 **일의 자리 숫자**만 남겨요(예: 7월+8월=15 → 5끗). 0이 남으면 제일 낮은 "망통"이에요.

score(점수)는 숫자가 클수록 이기는 패예요. 나중에 컴퓨터는 이 숫자만 비교해서 누가 이겼는지 정해요. 마치 시험 점수가 높은 사람이 이기는 것과 같아요!

7. 입체 카드 마법사 - Renderer3D

419~555줄

이 부분은 **Three.js**라는 다른 회사가 만들어준 "3D 그림 도구"를 빌려서, 평평한 화면에 진짜 테이블과 카드가 둥둥 떠 있는 것처럼 보이게 만들어요.

7-1. 무대 준비하기 - init()

426~450줄

▶ 연극을 시작하기 전에 무대를 꾸미는 것과 똑같아요:

- **scene**: 텅 빈 무대(연극 배경)를 하나 만들어요.
- **camera**: "관객이 어디서 바라볼지" 카메라 위치를 정해요 (약간 위에서 아래로 내려다보게).
- **renderer**: 실제로 화면에 그림을 그려주는 "화가"예요. 그림자도 켜줘요.
- **ambLight, dirLight**: 무대 위에 조명을 두 개 켜요. 하나는 사방을 밝게, 하나는 한쪽 방향에서 비추는 조명이에요.
- **table**: 초록색 네모난 카드 테이블(전통 포포)을 만들어서 무대 바닥에 깔아요.
- 마지막 줄 **this.animate()**: "이제부터 계속 움직여!" 하고 시작 버튼을 눌러요.

7-2. 끊임없이 그림 다시 그리기 - animate()

452~456줄

▶ 만화 영화가 사실은 그림을 아주 빠르게 계속 바꿔서 움직이는 것처럼 보이는 것 알죠? 이 함수는 **1초에 수십 번씩** "지금 화면을 다시 그려줘!"를 스스로 반복해서 부르는 마법의 무한 반복 주문이에요.

7-3. 창문 크기가 바뀌면 - onWindowResize()

458~462줄

▶ 브라우저 창 크기를 늘리거나 줄이면, 카메라와 그림 크기도 그에 맞춰 다시 계산해줘요. 창문 크기가 바뀌면 커튼도 다시 맞추는 것과 같아요.

7-4. 카드에 그림 입히기 - getCardTexture / getBackTexture

464~477줄

▶ 카드 앞면·뒷면 그림을 한 번 만들면 **기억해뒀다가(textureCache)** 또 필요할 때 다시 만들지 않고 재사용해요. 매번 새로 그리면 느려지니까, "아까 그린 거 또 쓰자!" 하는 알뜰한 방법이에요.

7-5. 카드 모양 만들기 - createCardMesh()

479~493줄

▶ 납작한 상자(가로 0.72, 세로 1.08, 두께 0.05)를 하나 만들고, 앞면엔 카드 그림을, 뒷면엔 빨간 뒷면 그림을, 옆면엔 빨간 테두리 색을 입혀요. 마치 종이 상자에 스티커를 여섯 면에 붙이는 것과 같아요.

7-6. 판 정리하기 - clearBoard()

495~500줄

▶ 새 판이 시작되기 전에, 무대 위에 있던 카드와 동전(칩) 그림들을 **싹 다 치워요**. 놀이 끝나고 장난감을 정리함에 넣는 것과 같아요.

7-7. 카드 나눠주기 애니메이션 - dealCard()

502~519줄

▶ 카드를 화면 오른쪽 구석(3.2, 0, 0.5)에서 시작해서, 목표 자리까지 **0.55초 동안 스르륵 미끄러지듯** 움직이게 해요(TWEEN 사용). 카드를 뒤집어서 보여줘야 하면 (`isFaceUp`) 회전도 같이 시켜요. 카드가 도착할 즈음엔 "탁" 소리도 내줘요.

7-8. 카드 뒤집기 - flipCards()

521~526줄

▶ 쇼다운(승부 확인) 순간에 상대방 카드를 0.45초 동안 부드럽게 뒤집어서 보여줘요.

7-9. 동전(칩) 쌓기 - addChips()

528~544줄

▶ 판돈이 오갈 때마다 동전 모양(원기둥) 여러 개를 무작위 위치에 만들어서, 통통 튀며 판 위에 쌓이는 것처럼 보이게 해요. 돈이 많을수록(`amount` 가 클수록) 동전 개수가 늘어나요(최소 2개, 최대 10개).

8. 소리를 만드는 마법사 - SoundFX

558~625줄

이 게임은 소리 파일(mp3 등)을 **하나도 쓰지 않아요!** 대신 컴퓨터가 "빠-" "웅-" 하는 순수한 전자 신호를 그때그때 직접 만들어서 소리를 내요. 마치 입으로 악기 소리를 흉내 내는 것과 비슷해요.

8-1. 기본 도구 - `tone()`, `noise()`

565~588줄

▶ **`tone()`**은 "빠-" 하는 순수한 음(音) 하나를 원하는 높낮이·길이·모양(사인파/삼각파)으로 만들어요. 피아노 건반 하나를 누르는 것과 같아요.
`noise()`는 "쌩아-" 하는 잡음(백색소음)을 만들어요. 카드가 착 놓일 때 나는 마찰음 같은 걸 만들 때 써요.

8-2. 진짜 효과음들 - `cardPlace`, `chips`, `win`, `lose`

590~614줄

▶ **`cardPlace()`**: 카드를 탁자에 놓을 때 나는 "탁" 소리 (잡음 + 낮은 음 조합).
`chips(amount)`: 판돈 액수에 따라 "짚랑짚랑" 동전 소리를 여러 번 내요. 돈이 많을수록 소리가 더 많이 나요.
`win()`: 이겼을 때 "딩동댕~▶" 하고 올라가는 밝은 멜로디를 연주해요.
`lose()`: 졌을 때 "뚜... 뚜... 뚜웅" 하고 내려가는 슬픈 멜로디를 연주해요.

9. 게임의 진짜 두뇌 - Game

627~1051줄

여기가 **제일 중요한 부분**이에요! "돈이 얼마 남았는지", "지금 몇 판째인지", "다음에 뭘 해야 하는지" 다 여기서 결정해요. 이 게임의 진짜 뇌라고 생각하면 돼요.

9-1. 게임이 기억해두는 것들 - 상태 변수

628~640줄

```
const Game = {
  state: 'INIT',
  userBalance: 100000,
  potMoney: 0,
  baseBet: 1000,
  roundBaseBet: 1000,
  aiRaiseExtra: 0,
  aiStyle: 'balanced',
  ...
};
```

▶ 이걸 게임이 항상 기억하고 있는 "메모지"예요.

- **state**: 지금 게임이 시작 화면인지(INIT) 게임 중인지(IN_GAME)
- **userBalance**: 내가 가진 돈 (처음엔 100,000원)
- **potMoney**: 지금 테이블 위에 쌓인 판돈
- **baseBet / roundBaseBet**: 기본 판돈 (내가 고른 것 / 이번 판에 딱 고정된 것)
- **aiRaiseExtra**: 상대가 얼마를 더 얹었는지(재레이즈했는지) 기억하는 곳
- **aiStyle**: 상대 컴퓨터의 성격(소심함/균형/공격적/블러핑쟁이)

9-2. 게임 켜지자마자 - init()

642~646줄

▶ 3D 무대를 켜고(Renderer3D.init), 족보표를 그리고(JokboBoard.render), 버튼들이 눌리면 반응하게 연결하고(bindEvents), 화면 숫자들을 최신으로 맞춰요(updateUI). "게임 켜자마자 해야 할 일 목록"이에요.

9-3. 기본 판돈 정하기 - setBaseBet, applyBaseBet

648~697줄

▶ 드롭다운에서 판돈을 고르면(setBaseBet), "커스텀"을 골랐는지 확인하고, 숫자가 이상하지 않은지, 내가 가진 돈보다 크지 않은지 확인해요. 문제 없으면 **applyBaseBet**이 실제로 "다음 판 기본 판돈"으로 저장해줘요. 만약 리스트에 없는 숫자를 직접 입력했으면, 선택 상자에 그 숫자를 **새로운 항목으로 몰래 추가**해줘요 (686~692줄) — 마치 메뉴에 없는 음식을 새로 적어넣는 것과 같아요.

9-4. 잔고 부족 안내 - showInsufficientBalance, resetBalanceIfInsufficient

660~682줄

▶ 돈이 모자라면 "잔고가 부족합니다" 하고 알려줘요. 특별히 **정확히 0원**이 되면, 불쌍하니까 자동으로 100,000원을 다시 채워주고 기본 판돈도 1,000원으로 되돌려 줘요 — 마치 용돈이 다 떨어지면 부모님이 다시 채워주는 것과 비슷해요.

9-5. 커스텀 판돈 팝업 다루기

699~725줄

▶ **openCustomBetDialog**: 팝업창을 열고 입력칸에 자동으로 커서를 놓아줘요.

closeCustomBetDialog: 팝업창을 닫아요.

confirmCustomBet: 사용자가 쓴 글자에서 숫자만 쏙 뽑아내고(replace(/^[^0-9]/g, '')), 1원보다 작거나 내 돈보다 많으면 에러 메시지를 보여주고, 괜찮으면 적용해요.

9-6. 버튼 연결하기 - bindEvents

727~735줄

▶ "적용" 버튼을 누르면 confirmCustomBet을, "취소"를 누르면 closeCustomBetDialog를 실행하도록 **선(연결줄)을 이어줘요**. 또, ESC 키를 누르면 팝업을 닫거나 시작화면으로 돌아가게 해줘요.

9-7. 버튼 글씨 바꾸기

741~756줄

▶ 평소엔 버튼에 "콜", "하프"라고 써있지만, 상대가 레이즈를 하면 버튼 글씨를 "콜 15,000원", "재레이즈 하프"처럼 **상황에 맞게 바뀌워요**. 마치 신호등이 상황에 따

라 색을 바꾸는 것과 같아요.

9-8. 상대방이 하는 말 고르기 - getAiPsychMessage 758~766줄

▶ "다이했을 때", "콜했을 때", "레이즈했을 때" 각각 여러 개의 대사 후보 중에서 **랜덤으로 하나**를 뽑아줘요. 매번 똑같은 말만 하면 재미없으니까, 여러 대사를 준비해 뒀다가 주사위 굴리듯 골라요.

9-9. 화면 숫자 새로고침 - updateUI, showToast, showPsychMessage 768~811줄

▶ **updateUI**: 내 잔고, 판돈 숫자를 화면 글씨에 다시 적어 넣어요.
showToast: 가운데에 잠깐 뜨는 알림 글씨를 보여줬다가 2초 뒤에 사라지게 해요.
showPsychMessage: 상대방 말풍선을 부드럽게 나타나게(또는 새 말로 바꾸게) 해줘요.

9-10. 시작 화면으로 돌아가기 - returnToStartScreen 821~828줄

▶ 게임 화면을 숨기고 다시 처음 시작 화면을 보여줘요. 놀이터에서 집으로 돌아가는 것과 같아요.

9-11. ☆ 새 판 시작하기 - startNewRound 830~899줄

NEW 이 함수는 "자, 새 게임 한 판 시작하자!"를 담당해요. 순서대로:

1. 내 돈이 **정확히 0원**이면 100,000원으로 채워주고 판돈도 1,000원으로 초기화해요.
2. 이번 판에 쓸 기본 판돈(`roundBaseBet`)을 확정해요. 돈이 모자라면 게임을 시작 안 하고 멈춰요.
3. 화면을 게임 화면으로 바꾸고, 상대(AI)의 성격을 **내 가지 중 무작위로** 골라요 (tight/balanced/aggressive/bluffer).
4. 테이블을 깨끗이 치우고, 판돈을 0으로 리셋해요.
5. **엔티(ante)**: 내 돈에서 기본 판돈만큼 빼고, 판돈에는 그 **두 배**를 더해요 (내가 낸 것 + 상대가 낸 척하는 것).
6. 카드 20장을 뒤죽박죽 섞어서(`sort(() => Math.random() - 0.5)`), 앞 2장은 내 패, 다음 2장은 상대 패로 나눠줘요.
7. 내 카드 두 장을 화면에 부드럽게 배달해주고(dealCard), 상대 카드도 뒤집힌 채로 배달해줘요.
8. 0.8초 뒤에 내 패 이름(예: "알리")을 화면에 보여줘요.

9-12. ☆☆☆ 내가 베팅하기 - handlePlayerBet 902~963줄

▶ 콜/하프/쿼터/올인/다이 버튼을 누르면 이 함수가 실행돼요. 여기서 **제일 중요한 갈림길**이 두 개 있어요:

① 다이를 눌렀다면 905~910줄

바로 "졌다"고 표시하고 1.5초 뒤에 새 판을 시작해요. 지금까지 낸 돈은 돌려받지 못해요.

② 상대가 이미 레이즈한 상태라면(`aiRaiseExtra > 0`) 914~942줄

먼저 상대가 올린 만큼(`required`)을 **정확히 맞춰야** 해요. 거기에 더해서 하프/쿼터/올인을 고르면 그만큼을 **더 얹어서(재레이즈)** 다시 상대에게 넘겨요. 그렇지 않고 그냥 맞추기만 했으면 승부(쇼다운)로 넘어가요.

③ 평범하게 처음 베팅한다면 945~962줄

콜=기본판돈, 하프=지금 판돈의 절반, 쿼터=4분의 1, 올인=내 돈 전부를 계산해서 내 돈에서 빼고 판돈에 더해요. 그 다음 **상대(AI) 차례**로 넘겨요 (`processAiTurn` 호출).

9-13. ☆☆☆ 상대(AI)가 생각하기 - processAiTurn 965~1020줄

▶ 이건 컴퓨터 친구가 **속으로 생각하는 과정**을 흉내 낸 부분이에요. 사람처럼 진짜로 생각하는 게 아니라, 정해진 계산식과 **주사위(랜덤 숫자)**로 결정해요.

1. **내 패가 얼마나 센지 점수 매기기** (966~968줄): 0(제일 약함)~1(제일 강함) 사이 숫자로 바뀌요.
2. **포기(다이)할지 정하기** (971~985줄): 내 패가 약할수록(꼴 패) 포기 확률이 훨씬 높아요(52%). 강하면 아주 조금만 포기해요(6%). 성격이 "소심함(tight)"이면 더 잘 포기하고, "공격적/블러핑쟁이"면 덜 포기해요. 주사위를 굴려서(`Math.random()`) 그 확률보다 낮게 나오면 **진짜 포기**하고, 판돈을 통째로 나(플레이어)한테 줘요.
3. **더 올릴지(레이즈) 정하기** (987~1009줄): 포기하지 않기로 했으면, 이번엔 "더 크게 걸어볼까?"를 또 주사위로 정해요. 패가 강할수록 레이즈 확률이 높아지고, 성격이 "공격적"이거나 "블러핑쟁이"면 더 자주 크게 걸어요. 레이즈하기로 하면 얼마를 더 얹을지 계산해서 판돈에 더하고, 나에게 "얼마를 더 내야 하는지" 알려 줘요.
4. **레이즈도 안 하면** (1012~1019줄): 그냥 내가 낸 만큼 **똑같이 따라(콜)**하고, 승부(쇼다운)로 넘어가요.

▶ 상대 카드를 뒤집어서 보여준 다음, **SeotdaEngine.evalHand**로 매긴 점수를 비교해요:

- 내 점수가 더 크면 → "승리!" 소리와 함께 판돈을 전부 내가 가져가요.
- 내 점수가 더 작으면 → "패배..." 소리가 나고 아무것도 못 받아요.
- 점수가 똑같으면 → "무승부, 판돈 반씩!" 하고 판돈의 절반만 받아요.

2.5초 뒤에 자동으로 다음 판을 시작해줘요.

10. 마지막 마무리와 보너스 코드 1053~1197줄

10-1. 게임 켜기 버튼 1053줄

```
window.onload = () => Game.init();
```

▶ "웹페이지가 다 로딩되면, Game.init()을 실행해줘!" 라는 뜻이에요. 이 한 줄이 사실상 **전체 게임의 시작 스위치**예요.

10-2. AI 말풍선 위치를 나중에 한 번 더 손보는 코드 1056~1119줄

▶ 이건 "패치(고침)" 코드예요. 원래 있던 AI 말풍선 위치 계산이 다른 상자들과 자꾸 겹쳐서, 나중에 따로 **덧붙여서 다시 계산**하는 코드를 추가한 거예요. 화면 크기가 바뀔 때마다(resize), "판돈 상자"와 "잔고 상자" 사이의 빈 공간을 실제로 재보고(`getBoundingClientRect`), 그 사이에 말풍선을 딱 맞게 끼워 넣어요. 공간이 너무 좁으면 아예 다른 줄(아래)로 옮겨줘요. 마치 책상 위에 물건이 너무 많아서 자로 빈 곳을 재고 새 물건을 끼워 넣는 것과 같아요.

10-3. 더 예쁜 동전 소리로 덮어쓰기 1062~1069줄, 1121~1153줄, 1178~1197줄

▶ 게임 뒷부분에 **비슷한 동전 소리 코드가 세 번**이나 다시 나와요. 이걸 앞에서 만든 소리를 "더 그럴듯한 금속 소리"로 **덮어쓰기(overwrite)**한 흔적이에요. 마치 그림을 그렸다가 마음에 안 들어서 그 위에 새 그림을 덧그린 것과 같아요. 가장 마지막에 정의된 버전이 실제로 사용돼요.

▶ **정리하면:** 이 파일은 "예쁜 옷 입히기(CSS)" → "무대와 소품 준비(HTML+Three.js+SoundFX)" → "규칙을 아는 심판(SeotdaEngine)" → "모든 걸 지휘하는 감독(Game 객체)" 순서로 이루어진 하나의 큰 인형극이에요! 그리고 감독(Game) 안에서도 **startNewRound → handlePlayerBet → processAiTurn → showdown** 이 네 함수가 한 판, 한 판을 돌아가게 만드는 진짜 심장이예요.

본 해설서는 3D_셋다.html(1,197줄) 전체를 기능 단위 블록으로 나누어 설명한 것입니다. 극히 반복적인 CSS 미디어쿼리, 3번 중복된 사운드 코드 등은 대표 사례 위주로 묶어 설명했으며, 실제 코드와 대조하려면 각 소제목 옆의 줄 번호를 참고하세요.